Computer Organization & Architecture

C Program Collection

Contents

1. Binary Adder (Sum and Carry)
2. Booth's Multiplication Algorithm
3. Memory Access Latency With and Without Cache
4. Memory-Mapped I/O Simulation
5. Interrupt-Driven I/O Simulation
6. Simulation of a 5-Stage Pipeline
7. Pipeline Hazard Detection and Resolution

Programs are ordered by topic: arithmetic, memory, input/output, then pipelining.

1. Binary Adder (Sum and Carry)

Adds two numbers and separates the 4-bit sum from the carry using a mask (& 15) and a shift (>> 4). Written for Turbo C.

Program

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int A, B, Total, Sum, Carry;

    clrscr();

    A = 5;
    B = 3;

    Total = A + B;
    Sum   = Total & 15;
    Carry = Total >> 4;

    // %04d pads the integer with leading zeros
    printf("A = %04d (binary)\n", A);
    printf("B = %04d (binary)\n", B);
    printf("SUM = %04d (binary)\n", Sum);
    printf("CARRY = %d\n", Carry);

    getch();
}
```

Output

```
A = 0005 (binary)
B = 0003 (binary)
SUM = 0008 (binary)
CARRY = 0
```

Note: %04d prints decimal digits padded with zeros; it does not convert to binary. To show true binary, print each bit with (value >> i) & 1.

2. Booth's Multiplication Algorithm

Multiplies 3 (0011) by 2 (0010) using Booth's algorithm and prints the 8-bit product.

Program

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int A=0,M,Q,Q1=0,i,b,p;
    char m[]="0011",q[]="0010";

    M=strtol(m,0,2);
    Q=strtol(q,0,2);
    for(i=0;i<4;i++)
    {
        if((Q&1)&&!Q1) A-=M;
        else if(!(Q&1)&&Q1) A+=M;
        b=A&1; Q1=Q&1;
        Q=(Q>>1)|(b<<3); A>>=1;
    }
    p=((A&15)<<4)|(Q&15);
    for(i=7;i>=0;i--) printf("%d",(p>>i)&1);
}
```

Output

```
00000110
```

Note: #include <stdlib.h> is required for strtol().

3. Memory Access Latency With and Without Cache

Computes average access time using: time = cache + (1 - hit) x ram, with a 10 ns cache, 100 ns RAM and 90% hit ratio.

Program

```
#include <stdio.h>
int main()
{
    int cache = 10, ram = 100;
    float hit = 0.9, time;

    time = ram;
    printf("Without Cache: %d ns\n", ram);

    time = cache + (1-hit)*ram;
    printf("With Cache: %.0f ns\n", time);

    return 0;
}
```

Output

```
Without Cache: 100 ns
With Cache: 20 ns
```

4. Memory-Mapped I/O Simulation

Simulates memory-mapped I/O by writing 1 (ON) and 0 (OFF) to the LED's address, 0x7700. Written for Turbo C.

Program

```
#include <stdio.h>
#include <conio.h>

#define LED_ADDRESS 0x7700

void main()
{
    unsigned char memory[65536] = {0};

    clrscr();
    printf("Memory-mapped I/O Simulation\n\n");

    memory[LED_ADDRESS] = 1;
    printf("Writing 1 to address: 0x7700\n");
    printf("LED status: ON\n\n");

    memory[LED_ADDRESS] = 0;
    printf("Writing 0 to address: 0x7700\n");
    printf("LED status: OFF\n\n");

    memory[LED_ADDRESS] = 1;
    printf("Writing 1 to address: 0x7700\n");
    printf("LED status: ON\n");

    getch();
}
```

Output

```
Memory-mapped I/O Simulation

Writing 1 to address: 0x7700
LED status: ON

Writing 0 to address: 0x7700
LED status: OFF

Writing 1 to address: 0x7700
LED status: ON
```

Note: 1 = ON, 0 = OFF, and the LED address is 0x7700. A 64 KB array stands in for the memory space.

5. Interrupt-Driven I/O Simulation

The CPU runs tasks 1 to 10; at task 5 an interrupt occurs, the ISR runs, and control returns to the main program.

Program

```
#include <stdio.h>
#include <dos.h>
#include <unistd.h>

void ISR()
{
    printf("\n*** Interrupt Received ***\n");
    printf("Executing Interrupt Service Routine...\n");
    printf("I/O Operation Completed\n");
    printf("Returning to Main Program...\n\n");
}

int main()
{
    int i;

    printf("Interrupt-Driven I/O Simulation\n\n");

    for(i = 1; i <= 10; i++)
    {
        printf("CPU executing task %d\n", i);

        sleep(1);

        if(i == 5)
            ISR();
    }

    return 0;
}
```

Output

```
Interrupt-Driven I/O Simulation

CPU executing task 1
CPU executing task 2
CPU executing task 3
CPU executing task 4
CPU executing task 5

*** Interrupt Received ***
Executing Interrupt Service Routine...
I/O Operation Completed
Returning to Main Program...

CPU executing task 6
CPU executing task 7
CPU executing task 8
CPU executing task 9
CPU executing task 10
```

Note: dos.h is Turbo C / DOS specific; remove it when compiling with GCC (sleep() comes from unistd.h).

6. Simulation of a 5-Stage Pipeline

Prints a 5-stage pipeline (IF, ID, EX, MEM, WB) cycle by cycle for 3 instructions over 7 clock cycles.

Program

```
#include <stdio.h>

int main()
{
    char *s[] = {"IF", "ID", "EX", "MEM", "WB"};
    int i, j;

    for(i = 0; i < 7; i++)
    {
        printf("Cycle %d:\t", i + 1);

        for(j = 0; j < 3; j++)
        {
            if(i >= j && i < j + 5)
                printf("%s\t", s[i - j]);
            else
                printf("\t");
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
Cycle 1:  IF
Cycle 2:  ID     IF
Cycle 3:  EX     ID     IF
Cycle 4:  MEM    EX     ID
Cycle 5:  WB     MEM    EX
Cycle 6:         WB     MEM
Cycle 7:                WB
```

Note: Each column is one instruction; each row is one clock cycle. Instruction j enters the pipeline j cycles after instruction 0.

7. Pipeline Hazard Detection and Resolution

Compares each instruction's destination register with the next instruction's source register. A match is a data hazard: a stall for the load instruction (I1), forwarding for the rest.

Program

```
#include <stdio.h>
#include <string.h>

int main()
{
    char *d[] = {"R1","R3","R5","R7"};
    char *s[] = {"R2","R1","R3","R5"};
    int i, f=0, st=0;

    printf("I1: LWR R1,R2\n");
    printf("I2: ADD R3,R1,R4\n");
    printf("I3: SUB R5,R3,R6\n");
    printf("I4: MUL R7,R5,R8\n");

    printf("\nHazard Detection and Resolution\n");

    for(i=1; i<4; i++)
    {
        if(strcmp(d[i-1],s[i])==0)
        {
            printf("Hazard on register %s\n",s[i]);

            if(i==1)
            {
                printf("Resolution: STALL inserted\n");
                st++;
            }
            else
            {
                printf("Resolution: FORWARDING applied\n");
                f++;
            }
        }
    }

    printf("\nTotal Forwardings: %d\n",f);
    printf("Total Stalls: %d\n",st);

    return 0;
}
```

Output

```
I1: LWR R1,R2
I2: ADD R3,R1,R4
I3: SUB R5,R3,R6
I4: MUL R7,R5,R8

Hazard Detection and Resolution
Hazard on register R1
Resolution: STALL inserted
Hazard on register R3
Resolution: FORWARDING applied
Hazard on register R5
Resolution: FORWARDING applied

Total Forwardings: 2
Total Stalls: 1
```

Note: #include <stdio.h> and <string.h> were added at the top; strcmp() needs string.h.